TraceClaw Reviewer Supplement: Controlled Cohort Evidence

Anonymous Author(s)

October 2026

Audited system. OpenClaw v2026.7.1-2, commit `0790d9f593ad30c940ed93b5872a8cf6d6f3cf8c`, entry path Control UI / WebChat `chat.send`. All line numbers in this document refer to the unmodified upstream source at that commit, not to the instrumented tree used to collect the traces.

1 Reading Guide

This supplement condenses the source-level audit into the evidence a reviewer is most likely to need. It covers every Gateway stage G0–G18 for the detailed case R1, expands the source sites behind the main claims, follows execution through the post-G18 Agent Runtime ledger AR0–AR6, and summarizes the controlled cohort R1–R7. The Full Audit Artifact retains the per-stage pseudocode, flow diagrams, and step-by-step walkthroughs.

Evidence status (how a claim is known).

- Observed: the saved runtime trace or a native runtime event directly records the stage outcome or value.
- SourceDerived: pinned source code plus observed upstream values determine the outcome, but the saved trace emits no separate event for that internal decision.
- NotObserved: the item is relevant to the audit but absent from the saved trace. Such values are printed as `NOT RECORDED`.

Branch status (what happened to a source branch). Taken: R1 follows the cited branch. NotReached: the branch exists in source but R1 does not reach it. NotApplicable: the branch does not apply to R1's observed inputs. NotReached and NotApplicable are not evidence categories; they normally carry evidence status SourceDerived.

Outcome observation is not rationale visibility. A stage is *covered* when it has at least one source anchor and at least one concrete run value, and *observed* when its outcome is directly recorded at runtime. For R1, Gateway coverage is 19/19 and stage-outcome observation is 19/19. Neither number says that every internal predicate, payload field, or return object is recorded: rationale visibility is *partial* for all 19 stages (full 0/19, partial 19/19). The last column of Table S4 lists the gap for each stage.

Anchor granularity. Anchors are given at the granularity at which the audit pins them. A single line or a short range anchors one statement. Where the audit pins only an enclosing function or block, the row says "function range" or "block", and no statement-level claim is attached to it. Table S1 expands the short file names used in the anchors.

Table S1: Short file names used in source anchors.

Short name	Path in the pinned snapshot
`message-handler.ts`, `auth-context.ts`	`src/gateway/server/ws-connection/`
`auth.ts`, `server-methods.ts`, `role-policy.ts`, `operator-scopes.ts`, `method-scopes.ts`, `session-utils.ts`, `chat-input-sanitize.ts`	`src/gateway/`

Short name	Path in the pinned snapshot
chat.ts	src/gateway/server-methods/
core-descriptors.ts	src/gateway/methods/
chat-send.ts	ui/src/pages/chat/
send-policy.ts, session-lifecycle-admission.ts	src/sessions/
agent-scope.ts	src/agents/
agent-events.ts	src/infra/
dispatch.ts	src/auto-reply/
dispatch-from-config.ts, inbound-context.ts, get-reply.ts, get-reply-run.ts, agent-runner.ts, agent-runner-execution.ts	src/auto-reply/reply/
attempt.ts	src/agents/embedded-agent-runner/run/
embedded-agent-subscribe.handlers.lifecycle.ts, embedded-agent-subscribe.handlers.tools.ts	src/agents/

2 Setting and R1 Identifiers

R1 is the detailed Gateway case. When the saved trace does not contain a field, the table says so rather than filling the value from a later layer or from source code.

Table S2: R1 setting.

Item	Recorded value
Target system	OpenClaw v2026.7.1-2 with the TraceClaw instrumentation patch applied
Source snapshot	0790d9f593ad30c940ed93b5872a8cf6d6f3cf8c
Entry path	Control UI / WebChat chat.send
User message	How to make a cake?
Authentication shape	shared-token path accepted; role operator; scope operator.write
Agent / runner	Agent main; runner embedded
Provider / model	openai / gpt-5.5
Available-tool inventory	NOT RECORDED (NotObserved)
Tools invoked in R1	0
Python, Node.js, package manager, operating system, hardware, browser build	not archived in the saved trace

Table S3: R1 identifiers and where they are obtained.

Variable	R1 value	Source / evidence
SESSION_KEY	agent:main:dashboard: trace-82fbff4483c4	extracted as rawSessionKey at chat.ts:3800; observed again in the G6/G7 runtime trace
SESSION_ID	NOT RECORDED	the Gateway Session ID is not emitted. Its read site is chat.ts:3848--3849. The Agent Runtime layer has its own Session identity, which is not used to fill this field
RUN_ID	ANON-R1-RUN-ID (anonymized)	generated by the Control UI (chat-send.ts:266--276); extracted as clientRunId at chat.ts:3802; observed in the G4 runtime trace
MESSAGE	"How to make a cake?"	validated parameters at chat.ts:3714--3739; sanitized into inboundMessage at chat.ts:3784

3 Gateway Ledger G0–G18

Table S4 has one row per Gateway stage. Every stage outcome in the table is Observed in the saved R1 trace. The branch label printed first in the third column is the label used in the generated stage-by-run matrix of the artifact; all seven controlled runs R1–R7 carry the same label at every stage. Values after the label are R1 values.

Table S4: Gateway ledger for R1. Stage outcomes are Observed; rationale visibility is partial at every stage.

Stage	Decision and source anchor	Branch label; R1 values	Rationale-level gap
G0	Resolve the connection-authentication state. Call message-handler.ts:941--957; function auth-context.ts:126--186.	authorized. Event connection_auth_state_resolved: authMode="token", authMethod="token", hasDeviceIdentity=False, sharedAuthProvided=True, sharedAuthOk=True.	Raw credential fields are not recorded; the effective device-token candidate and its source are source-derived as undefined.
G1	Initial shared-credential authorization. Call auth-context.ts:144--153; core logic auth.ts:481--584; token check auth.ts:389--411.	allow. Event shared_credential_authorized: authMode="token", authMethod="token", result="allow".	Exact credential comparison details and rate-limit internals are not emitted.
G2	Final connection authentication and identity. Call message-handler.ts: 1215--1252; decision function auth-context.ts:188--330.	pass. Event connection_authentication_completed: authMethod="token", role="operator", scopes=[operator.write], result="pass".	The device-signature subpath is source-derived because hasDeviceIdentity=False.

continued on next page

Stage	Decision and source anchor	Branch label; R1 values	Rationale-level gap
G3	Method authorization for `chat.send`. Call site `server-methods.ts:734--738`; function `server-methods.ts:262--299`.	`allow`. G3 event records the result; inputs `role="operator"`, `scopes=[operator.write]`.	`auth_reason` is not emitted; the administrator-shortcut exclusion is source-derived from the scopes.
G4	Validate the `chat.send` request envelope. Block `chat.ts:3699--3769`.	`pass`. G4 event: `has_privileged_fields=False`, `has_explicit_origin=False`.	The full raw WebSocket frame and all pre-validation fields are not archived.
G5	Sanitize and classify the message. Block `chat.ts:3770--3803`; sanitizer `chat-input-sanitize.ts:16--24`.	`unchanged`. G5 event: `message_length=19`, `message_changed_by_sanitization=False`.	The intermediate sanitizer object is not emitted in full.
G6	Resolve an explicitly requested Agent. Call `chat.ts:3804--3808`; function `chat.ts:725--745`.	`resolved`. `requested_agent_id=None`: no explicit override, and the Session key does not resolve to the global store.	The absence of every alternate override source is partly source-derived.
G7	Resolve the Session key and look up the store. Wrapper `chat.ts:3810--3824`; `loadSessionEntry` `session-utils.ts:1047--1068`.	`resolved`. The runtime event records `rawSessionKey` and the unchanged `canonicalSessionKey`.	Store keys, persistent store path, the loaded entry object, `entry.sessionId`, and timing are not emitted as runtime fields.
G8	Validate Agent / Session consistency. Call `chat.ts:3839--3847`; function `chat.ts:690--723`.	`pass`. G8 event records the result.	The full validation object and the backing Session identity are not emitted.
G9	Resolve the effective Agent. Call `chat.ts:3864--3868`; wrapper `agent-scope.ts:323--330`; precedence logic `agent-scope.ts:300--321`.	`resolved`. G9 event: `agent_id="main"`, `result="resolved"`.	The fallback candidate chain is not emitted in full.
G10	Evaluate the Session send policy. Call and deny gate `chat.ts:3891--3905`; `resolveSendPolicy` `send-policy.ts:76--159`.	`allow`. G10 event: `send_policy="allow"`, `result="allow"`.	The exact allow sub-branch and the rule rationale are not emitted.
G11	Idempotency and existing-run guard. Guards `chat.ts:3936--3942` through `chat.ts:4001--4005`.	`pass`. G11 event: `dedupeDecision="new_dispatch"`, `result="pass"`.	Internal lock / cache state is not emitted.
G12	Session work admission and revalidation. Reservation `chat.ts:4025--4047`; admission call `chat.ts:4067--4163`.	`admitted`. G12 event: `admissionDecision="admitted"`, `result="admitted"`, `latestSessionRevalidated=true`.	Admission internals, latest Session details, and the concrete admitted Session identity are not emitted.

continued on next page

Stage	Decision and source anchor	Branch label; R1 values	Rationale-level gap
G13	Construct the runtime message context. `MsgContext` construction `chat.ts:4503--4545`.	`constructed`. Event `runtime_context_constructed` records `agentId="main"`, `chatType="direct"`, `hasMedia=false`, and `result="constructed"`. The `operator.write` scope is carried into the context at `chat.ts:4544` from the already observed client scopes.	The complete constructed context, including the full available-tool inventory, is not emitted.
G14	Enter inbound message dispatch. Call `chat.ts:4788--4797`; function `dispatch.ts:528--584`.	`entered`. Event `dispatch_inbound_entered`: `result=entered`, run ID, Session key, `agentId=main`. The native diagnostic span is not separately archived.	The complete dispatch return payload is not emitted.
G15	Finalize the inbound context. Call `dispatch.ts:544--551`; function `inbound-context.ts:76--175`.	`finalized`. Three `inbound_context_finalized` events with `result=finalized`.	The full finalized context object is not emitted.
G16	Reply dispatch orchestration. Function entry `dispatch-from-config.ts:1215--1217`; successful return `dispatch-from-config.ts:4051--4062`.	`normal_path`. Event `reply_dispatch_orchestration_entered`: `result=entered`. The G17 and G18 events that follow show that the normal path is traversed.	The complete reply-dispatch return object is not emitted.
G17	Re-resolve the effective Agent downstream. `dispatch-from-config.ts:1422--1427`.	`resolved`. Event `effective_agent_reresolved`: `agentId="main"`, `downstreamAgentId="main"`, `result="resolved"`.	The fallback attempt chain is not emitted in full.
G18	Select and invoke the reply resolver. Selection `dispatch-from-config.ts:3381--3384`; invocation `dispatch-from-config.ts:3389--3811`.	`selected`. Event `reply_resolver_selected`: `resolverSource="default_getReplyFromConfig"`, `agentId="main"`, `result="selected"`.	The full resolver payload and the later Gateway post-return state are not emitted.

Because the seven runs share one label at every stage, the Gateway ledger does not separate the runs from each other. The differences between runs appear after G18, in the Agent Runtime layer (Section 7).

4 Key Stages at Statement Level

The paper's main claims depend on a small set of decision sites. The tables below show those sites at statement level and pair each one with the R1 outcome. A row marked SourceDerived is a branch conclusion obtained from pinned source and observed inputs, not a separately emitted event.

4.1 G0: Authentication-State Resolution

G0 gathers the authentication material and returns a `ConnectAuthState`. It does not make the final connection decision. Three objects must be kept apart for the device token (Table S5).

Table S5: G0 device-token objects for R1.

Object	R1 value	Evidence	Basis
Raw handshake field `connectAuth.deviceToken`	`NOT RECORDED`	NotObserved	Raw credential material is not archived.
Effective program state `deviceTokenCandidate` and `deviceTokenCandidateSource`	undefined	SourceDerived	R1 records `hasDeviceIdentity=False`; `auth-context.ts: 141--142` then selects the empty-object branch. Candidate resolution (`auth-context.ts: 111--124`) is NotReached.
Emitted event field `hasDeviceTokenCandidate`	absent	NotObserved	Not among the R1 event fields. Its absence does not make the effective state unknown.

Table S6: G0 statements and R1 values.

Statement	Anchor	R1	Evidence
Call `resolveConnectAuthState`	`message-handler.ts: 941--957`	reached	Observed
Resolve the shared-credential candidate	`auth-context.ts:136`, `auth-context.ts:137`; helper `auth-context.ts: 100--109`	`sharedAuthProvided=True`	Observed
Resolve the bootstrap-token candidate (only with a device identity)	`auth-context.ts: 138--140`	undefined	SourceDerived
Resolve the device-token candidate (only with a device identity)	`auth-context.ts: 141--142`	undefined; helper NotReached	SourceDerived
Nested G1 authorization call	`auth-context.ts: 144--153`	`auth_result.ok=True`; G1 `result="allow"`	Observed
Shared-auth probe	`auth-context.ts: 155--166`	reached, because a shared credential exists	SourceDerived
Compute `sharedAuthOk`	`auth-context.ts: 170--173`	`True`	Observed
Return `ConnectAuthState`	`auth-context.ts: 175--185`	`authOk=True`, `authMethod="token"`, `result="authorized"`	Observed

4.2 G2: Final Authentication Decision

G2 receives the G0 state and decides the connection. For R1 the shared-token result from G1 is carried through; no device identity is supplied, so the device-identity checks do not apply and the device-token fallback is not reached.

The core decision function spans `auth-context.ts:208--330`, but the rows below cite the statements that actually govern R1. The full-function range is retained only as context; it is not used in place of the narrower branch anchors.

Table S7: G2 statements and R1 values.

Statement / branch	**Anchor**	**R1**	**Evidence**
Missing-device policy	`message-handler.ts: 1038--1136` (block)	`hasDeviceIdentity=False`; rejection not taken	SourceDerived
Device identity and signature validation	`message-handler.ts: 1137--1213` (block)	NotApplicable: no device identity	SourceDerived
Call `resolveConnectAuthDecision`	`message-handler.ts: 1215`	inputs `authOk=True`, `authMethod="token"`, `role="operator"`, `scopes=[operator.write]`	Observed
Serialization wrapper	`auth-context.ts: 188--207`	direct core path taken	SourceDerived
Initialize decision state	`auth-context.ts: 211--215`	shared-token state from G0 is carried into the core decision	SourceDerived
Construct the returned decision with `finish()`	`auth-context.ts: 217--227`	helper used by the successful early return	SourceDerived
Bootstrap-token candidate and branch	`auth-context.ts: 229--230`; rate-limit `auth-context.ts: 235--252`; verification `auth-context.ts: 253--276`	not taken: no device identity and no bootstrap-token candidate	SourceDerived
Read the device-token candidate and test the early-return guard	`auth-context.ts: 279--282`	Taken: `hasDeviceIdentity=False`; shared authentication has already succeeded	SourceDerived
Device-token rate limit and verification	`auth-context.ts: 284--299`; `auth-context.ts: 300--327`	NotReached; a device-token verification result is NotApplicablefor R1	SourceDerived
Final fallback return	`auth-context.ts:330`	not reached because R1 returns at the earlier guard	SourceDerived
Accept or reject the decision	`message-handler.ts: 1215--1252` (block)	reject branch not taken	Observed
Gateway-auth generation revalidation	`message-handler.ts: 1286--1295`	no generation-mismatch close	SourceDerived
Pairing and approved role / scope validation	`message-handler.ts: 1335--1757` (block)	final role `operator`, scopes `[operator.write]`; exact pairing sub-branch `NOT RECORDED`	Observed (values); NotObserved (sub-branch)
Register the authenticated client	`message-handler.ts: 2056--2065`	`isDeviceTokenAuth=False`	SourceDerived

Statement / branch	Anchor	R1	Evidence
Build and send `hello-ok`	`message-handler.ts:2169--2204`, `message-handler.ts:2237--2273`	event `connection_authentication_completed`, `result="pass"`	Observed

Claim limit. `authMethod="token"` is the shared-token method. It is not evidence of device-token authentication. The device-token rate-limit guard and issuer generation are not recorded for R1.

4.3 G3: Method Authorization

G3 decides whether the authenticated client may invoke `chat.send`. Two nearby routes matter here: an administrator shortcut and the ordinary required-scope check. R1 carries `scopes=[operator.write]`, so the shortcut does not apply and the required-scope check is reached.

Table S8: G3 statements and R1 values. All anchors are in `server-methods.ts` unless another file is named.

Statement / branch	Anchor	R1	Evidence
Call `authorizeGatewayMethod`	`server-methods.ts:734`	`method="chat.send"`	Observed
Error response if authorization fails	`server-methods.ts:735--738`	not taken	SourceDerived
Pre-connect shortcut	`server-methods.ts:270--272`	not taken: `client.connect` is present	SourceDerived
`health` shortcut	`server-methods.ts:273--275`	not taken	SourceDerived
Read and parse the role	`server-methods.ts:276`, `server-methods.ts:277`	`role="operator"`	Observed
Read the scopes	`server-methods.ts:281`	`scopes=[operator.write]`	Observed
Role authorization for the method	`server-methods.ts:282--284`; rule `role-policy.ts:24--29`	pass: `chat.send` is an operator RPC	SourceDerived
Node shortcut	`server-methods.ts:285--287`	not taken	SourceDerived
Administrator shortcut (`operator.admin` in scopes)	`server-methods.ts:288--290`; constant `operator-scopes.ts:3`	not taken: `operator.admin` is absent from the observed scopes	SourceDerived
Look up the registered scope	`server-methods.ts:291`; `core-descriptors.ts:231`	reached; registered scope is `operator.write`	SourceDerived
Required-scope check	`server-methods.ts:292--294`; helper `method-scopes.ts:257--274`	reached	SourceDerived

Statement / branch	Anchor	R1	Evidence
Missing-scope rejection	`server-methods.ts:295--297`	not taken	SourceDerived
Return without error	`server-methods.ts:298`	`auth_error=None`	SourceDerived
G3 event	runtime	`auth_result="allow"`	Observed
Authorization reason	runtime	`auth_reason` NOT RECORDED	NotObserved

Claim limit. The trace shows the outcome and the scopes. The source shows which branch those scopes select. The cohort does not establish why R1 holds `operator.write` rather than another scope set, and the paper does not claim that the authentication method determines the scope.

4.4 G11: Idempotency and Existing-Run Guard

G11 checks the client run ID against the existing-run registries and the pre-admission guards. Any match would short-circuit the request. R1 reaches none of those exits and proceeds as a new dispatch.

Table S9: G11 guards and R1 values.

Guard	Anchor	R1	Evidence
Completed result cached for this run ID	`chat.ts:3936--3942`	not taken	SourceDerived
Run previously aborted	`chat.ts:3944--3965`	not taken	SourceDerived
Pending pre-registered run (in flight)	`chat.ts:3967--3978`	not taken	SourceDerived
Active run with the same ID (in flight)	`chat.ts:3980--3987`	not taken	SourceDerived
Queued turn with the same ID (in flight)	`chat.ts:3988--3994`	not taken	SourceDerived
Session routing changed	`chat.ts:3995--4000`	no short-circuit	SourceDerived
Session not allowed to start work (archived)	`chat.ts:4001--4005`	no short-circuit	SourceDerived
G11 event	runtime	`dedupeDecision="new_dispatch", result="pass"`	Observed

The pending-run key used by these guards is created earlier at `chat.ts:3803`. The individual registry lookups are not emitted; only the decision is.

4.5 G12: Work Admission and Revalidation

G12 moves the new-dispatch candidate into Session-scoped work. A pending reservation is created first; admission then revalidates the latest Session state, registers the abort controller, and checks for races before the run context is claimed.

Table S10: G12 statements. R1 follows the successful path through all of them.

Statement / check	Anchor	Failure branch
Capture the Agent lifecycle generation	`chat.ts:4025`; helper `agent-events.ts:195--197`	–
Create the pending reservation	`chat.ts:4030--4047`	–
Call `beginSessionWorkAdmission`	`chat.ts:4067--4163`; helper `session-lifecycle-admission.ts:327--378`	–
run already aborted	`chat.ts:4071--4073`	return
reservation superseded by another attempt	`chat.ts:4079--4085`	mark superseded
lifecycle generation changed	`chat.ts:4094--4103`	abort, reason `restart`
reservation missing or expired	`chat.ts:4104--4116`	abort, reason `timeout`
reload the latest Session	`chat.ts:4117`	–
Session routing changed	`chat.ts:4118--4120`	raise
Session deleted while starting	`chat.ts:4122--4124`	reject
backing Session identity changed	`chat.ts:4125--4132`	reject
Session archived	`chat.ts:4133--4136`	raise
register the abort controller	`chat.ts:4139--4155`	–
Clear the pending reservation	`chat.ts:4165--4172`	–
Reservation superseded (post-admission)	`chat.ts:4174--4188`	short-circuit
Lifecycle generation changed (post-admission)	`chat.ts:4189--4212`	abort and short-circuit
No abort controller	`chat.ts:4213--4225`	reject admission
Abort controller not newly registered	`chat.ts:4226--4235`	return existing in-flight run
Claim the Agent run context	`chat.ts:4240--4244`	–

Table S11: G12 values for R1.

Item	R1 value	Evidence
G12 event	`admissionDecision="admitted"`, `result="admitted"`, `latestSessionRevalidated=true`	Observed
`reservation_superseded`	`False`	SourceDerived
Abort controller exists and is registered	yes	SourceDerived
`backing_session_id`	`NOT RECORDED`	NotObserved
`admitted_session_id`	`NOT RECORDED` (the source can fall back to the client run ID)	NotObserved

Claim limit. `latestSessionRevalidated=true` is an event field. It supports that revalidation ran and admission succeeded. It is not a field-by-field comparison of the latest Session with the one loaded in G7.

4.6 G16–G18: Dispatch, Tool Policy, and Resolver Boundary

G16 is the orchestration layer that contains G17 and G18. Several branches can return before the default Agent runtime is entered. R1 does not take those exits: the observed G17 and G18 events show that execution reaches the downstream Agent re-resolution and resolver selection.

Table S12: G16 structure with the nested G17 and G18. All anchors are in `dispatch-from-config.ts`.

Statement / block	**Anchor**	**R1**
Function entry	`dispatch-from-config.ts: 1215--1217`	event `reply_dispatch_orchestration_entered`, `result=entered` (Observed)
Early-abort check	`dispatch-from-config.ts:1219`	execution continues (SourceDerived)
Initial Session-store lookup	`dispatch-from-config.ts:1233`; helper `dispatch-from-config.ts: 441--470`	–
Bound ACP dispatch Session: call	`dispatch-from-config.ts:1341`; helper definition `dispatch-from-config.ts: 473--502`	–
Dispatch Session keys	`dispatch-from-config.ts: 1342--1343`, `dispatch-from-config.ts: 1347--1348`	–
G17: re-resolve the Agent and load its configuration	`dispatch-from-config.ts: 1422--1427`	event `effective_agent_reresolved`: `agentId="main"`, `downstreamAgentId="main"` (Observed)
Reply-operation admission	`dispatch-from-config.ts: 1533--1623`; `admitReplyTurn` `dispatch-from-config.ts: 1609--1623`	–
Downstream send-policy evaluation	`dispatch-from-config.ts: 2184--2196`	–
Effective tool policy	`dispatch-from-config.ts: 2206--2210`	path SourceDerived; resolved value NotObserved
Message-tool availability	`dispatch-from-config.ts: 2289--2299`	–
Source-reply visibility policy	`dispatch-from-config.ts: 2300--2321`	–
Inbound dedupe claim; duplicate exit	`dispatch-from-config.ts:2342`; `dispatch-from-config.ts: 2343--2349`	exit not taken (SourceDerived)
Fast-abort resolver; early exit	`dispatch-from-config.ts: 2428--2474`	exit not taken (SourceDerived)

Statement / block	Anchor	R1
Pre-dispatch ownership; plugin-owned binding	`dispatch-from-config.ts:2478`; `dispatch-from-config.ts:2486--2633`	exit not taken (SourceDerived)
`before_dispatch` hooks	`dispatch-from-config.ts:2908--2968`	exit not taken (SourceDerived)
`reply_dispatch` hooks	`dispatch-from-config.ts:2973--3023`	exit not taken (SourceDerived)
Final dispatch ownership	`dispatch-from-config.ts:3025`	–
G18: select the resolver (supplied, else load the default)	`dispatch-from-config.ts:3381--3384`	event `reply_resolver_selected`: `resolverSource="default_getReplyFromConfig"` (Observed)
G18: prepare the reply configuration	`dispatch-from-config.ts:3385--3387`	–
G18: record Agent-dispatch start	`dispatch-from-config.ts:3388`	–
G18: invoke the resolver under lifecycle admission	`dispatch-from-config.ts:3389--3811` (block)	Agent Runtime runs; see Section 6
Resume: Session metadata changes	`dispatch-from-config.ts:3812`, `dispatch-from-config.ts:3813`	SourceDerived
Resume: normalize and send final payloads	`dispatch-from-config.ts:3886`; `dispatch-from-config.ts:3943`	SourceDerived
Resume: complete and return	`dispatch-from-config.ts:4040` to `dispatch-from-config.ts:4050`; return `dispatch-from-config.ts:4051--4062`	SourceDerived; final counts NotObserved

Tool policy and approval boundary. Two source sites carry tool restrictions toward the runtime. In G14, a runtime tool allow-list is propagated into the reply options (`dispatch.ts:69--80`, called at `dispatch.ts:538`). In G16, the effective tool policy is resolved from the configuration, the dispatch Session key, and the Agent ID (`dispatch-from-config.ts:2206--2210`). The audit has no recorded R1 value for either site, and the paper treats the effective tool-policy path as SourceDerived. Tool executions themselves are Observed in the Agent Runtime layer. No separate approval or confirmation event is recorded. These three statements are what the R3 finding in Section 7.3 relies on.

5 Remaining Stages: Decisive Statement and Nearest Branch Not Taken

For the stages not expanded above, Table S13 cites the statement that decides the R1 outcome and the nearest alternative that R1 does not take. The stage outcomes are Observed (Table S4); the branch conclusions in this table are SourceDerived unless marked otherwise.

Table S13: Decisive statements for G1, G4–G10, and G13–G15.

Stage	Statement that decides R1	Nearest branch not taken by R1
G1	Token-mode branch `auth.ts:562--570` calls the token check; the check returns success at `auth.ts:410`. R1: `auth.mode="token"` (Observed).	Trusted-proxy mode `auth.ts:490--531` and no-auth mode `auth.ts:533--535` are not taken. The token-mismatch return `auth.ts:405--408` is not taken. The rate-limit state at `auth.ts:537--540` is `NOT RECORDED`.
G4	Schema validation `chat.ts:3703`; privileged-field detection `chat.ts:3751--3755`. R1: no privileged fields and no explicit origin (Observed).	Invalid-parameters rejection `chat.ts:3703--3713`; invalid-origin rejection `chat.ts:3747--3749`; administrator-scope rejection `chat.ts:3757--3769`.
G5	Sanitize `chat.ts:3770`; classify stop / btw `chat.ts:3787`, `chat.ts:3788--3789`; extract Session key, Agent override, and run ID at `chat.ts:3800`, `chat.ts:3801`, `chat.ts:3802`. R1: main turn, no attachments.	Sanitizer rejection `chat.ts:3771--3778`; empty message with no attachment `chat.ts:3792--3799`.
G6	Non-global store returns no requested Agent `chat.ts:738--742`. R1: `parsed.agentId="main"` but the store key is not `global`, so `requested_agent_id=None`.	Explicit `agentId` override `chat.ts:731--733`; global-store return of the parsed Agent `chat.ts:744`.
G7	`loadSessionEntry` returns the canonical key (`session-utils.ts:1065`) and store keys (`session-utils.ts:1066`); both are Observed for R1.	Legacy-key match `session-utils.ts:1059`: R1 `legacy_key=None`. Store path and `entry_exists` are `NOT RECORDED`.
G8	With no requested Agent, validation returns success immediately at `chat.ts:696--698`.	Unknown Agent `chat.ts:699--701`; Agent / Session-key mismatch `chat.ts:705--708`; deleted-Agent rejection `chat.ts:3853--3863`.
G9	Precedence expression `agent-scope.ts:317--319`: explicit Agent, then Session-key Agent, then fallback, then default. R1 takes the Session-key Agent branch; `agent_id="main"` (Observed).	Explicit, fallback, and default Agent branches of the same expression.
G10	`resolveSendPolicy` is called at `chat.ts:3891--3897`; R1 `send_policy="allow"` (Observed). Which allow-producing branch fired is `NOT RECORDED`: Session override `send-policy.ts:84--86`, no configured policy `send-policy.ts:89--91`, matching allow rule `send-policy.ts:153--155`, or default `send-policy.ts:158`.	Deny gate `chat.ts:3898--3905`; ambiguous-key deny `send-policy.ts:92--96`; matching deny rule `send-policy.ts:147--149`.
G13	Context fields are assigned at `chat.ts:4510` (Session key), `chat.ts:4511` (Agent), `chat.ts:4519` (chat type), and `chat.ts:4544` (client scopes). The runtime event directly records `agentId="main"`, `chatType="direct"`, and `hasMedia=false`; `GatewayClientScopes=[operator.write]` is SourceDerived from the source assignment plus the already observed client scopes.	Slash-command turn `chat.ts:4522--4528`: R1 is an ordinary message turn. Staged-media fields `chat.ts:4548--4559`: R1 has no attachments.
G14	`dispatchInboundMessage` is called at `chat.ts:4788--4797`. Inside it, G15 is called at `dispatch.ts:544--551` and G16 at `dispatch.ts:562--582`; the result is reconciled and returned at `dispatch.ts:583`.	Result reconciliation for cancelled or failed deliveries `dispatch.ts:488--524`; the dispatch return payload is not emitted for R1.

Stage	Statement that decides R1	Nearest branch not taken by R1
G15	Finalizer `inbound-context.ts:76--175`: text normalization `inbound-context.ts:83--97`, chat type `inbound-context.ts:99--102`, command state `inbound-context.ts:135--143`. R1: message, Agent, and chat type are unchanged.	Media-type alignment `inbound-context.ts:145--173`: not needed, R1 has no attachments.

6 Post-G18 Agent Runtime (AR0–AR6)

The Gateway stage model stops at G18. What happens after the resolver is invoked is recorded in a separate `agentRuntime` object with stages AR0–AR6, not as additional Gateway stages.

Source path. The G18 resolver call enters `getReplyFromConfig`, then `runPreparedReply`, `runReplyAgent`, and `runAgentTurnWithFallback`, which takes either the CLI branch (`runCliAgentWithLifecycle`) or the embedded branch (`runEmbeddedAgent`). On the embedded branch the runtime resolves the Skills prompt, constructs runtime Tools, creates the Agent session, subscribes to lifecycle and tool events, prompts the model, runs the assistant / tool event loop, takes the winning `EmbeddedAgentRunResult`, finalizes the reply, ends the Agent lifecycle, and returns the reply result across G18. G16 then resumes, and G14 returns the final dispatch result.

Table S14: Post-G18 source anchors. Rows marked "function range" or "file" locate a boundary and carry no statement-level claim.

Boundary	Anchor
Resolver selection	`dispatch-from-config.ts:3381--3384`
Default resolver	`get-reply.ts` (file)
Prepared reply runner	`get-reply-run.ts:512--1641` (function range)
Reply Agent runner	`agent-runner.ts:1130--2753` (function range)
CLI / embedded runtime selection	`agent-runner-execution.ts:2338--2660`
Skills prompt construction	`attempt.ts:1150--1158`
Runtime tool construction	`attempt.ts:1342--1383`
Agent session creation	`attempt.ts:2557--2573`
Model prompt boundary	`attempt.ts:3582--3588`
Agent event subscription	`attempt.ts:3711--3749`
Agent lifecycle start / end	`embedded-agent-subscribe.handlers.lifecycle.ts:27--58,170--225`
Tool start / result	`embedded-agent-subscribe.handlers.tools.ts:1049--1059,1489--1502`
Global Agent event bus	`agent-events.ts:487--492`

Table S15: Agent Runtime ledger for R1.

AR	Meaning	Event	R1 value	Evidence
AR0	Runtime selected	`agent_runtime_selected`	runner `embedded`; Agent `main`; provider `openai`; model `gpt-5.5`	Observed
AR1	Agent run started	`agent_run_started`	correlated with the R1 run ID and Session key	Observed
AR2	Tool / no-tool outcome	`tool_started` / `tool_result`	no pair is present: observed `no_tool`	Observed
AR3	Reply finalized	`agent_reply_finalized`	final reply event present; text omitted here	Observed
AR4	Run ended	`agent_run_ended`	stop reason `stop`	Observed
AR5	Resolver returned	`reply_resolver_returned`	reply-result kind `single`	Observed
AR6	Return to Gateway post-processing	none	G16 / G14 continuation follows from source after the resolver return	SourceDerived

For a no-tool run, the absence of a tool event between an observed run start and an observed run end is itself the AR2 evidence; AR2 is not missing. AR6 is SourceDerived in every run because no event is emitted after the resolver return.

7 Controlled Cohort R1–R7

7.1 Runs

Table S16: Controlled cohort. Tool ratio is `tool_result` / `tool_started`. "AR obs." counts directly observed AR stages out of seven; the remaining stage is AR6. Every run has Gateway coverage 19/19 and observation 19/19.

Run	Request	Runtime class	Observed tools	Tool ratio	AR obs.	Provenance
R1	cake baseline	no-tool	none	n/a	6/7	post-hoc provenance reconstruction
R2	OpenTelemetry retrieval	read-only retrieval	`web_search` ×6	6/6	6/7	post-hoc provenance reconstruction
R3	redacted email action	external side effect via host application	`bash` ×5	5/5	6/7	post-hoc provenance reconstruction

Run	Request	Runtime class	Observed tools	Tool ratio	AR obs.	Provenance
R4	Fibonacci file creation	local workspace mutation	`bash` ×3 + `apply_patch`	4/4	6/7	post-hoc provenance reconstruction
R5	tracing / provenance explanation	no-tool	none	n/a	6/7	post-hoc provenance reconstruction
R6	tracing / provenance repeat	no-tool	none	n/a	6/7	fingerprinted
R7	cake repeat	no-tool	none	n/a	6/7	fingerprinted

Gateway branches across runs. All seven runs carry the same branch label at each of the 19 Gateway stages, and rationale visibility is partial at each stage in each run. The labels are the ones printed in Table S4. The rationale-visibility count is stored in `artifacts/controlled/rationale-visibility-summary.csv`. The cohort therefore contains one Gateway branch structure. It contains no rejected, retried, cancelled, or duplicate request.

Provenance. The saved R1–R5 traces do not embed the OpenClaw commit, the TraceClaw commit, a configuration fingerprint, or the instrumentation patch hash. Their environment provenance is reconstructed from the saved artifacts, repository history, and a post-hoc source/instrumentation audit. R6 and R7 add direct before-and-after fingerprints of the local checkout and instrumentation patch.

7.2 Exact-Prompt Repeats

Table S17: Repeatability pairs.

Pair	G0–G18 branch difference	Same Agent	Same resolver	Same tools	Same AR structure
R1 / R7	none	yes	yes	yes	yes
R5 / R6	none	yes	yes	yes	yes

Both pairs are no-tool runs. They test repeatability of the Gateway path and of the no-tool runtime path. They do not test repeatability of tool-using runs.

7.3 R3: External Side Effect

R3 asks the Agent to draft and send an email. The trace contains five `bash` tool calls.

Table S18: R3 tool calls in order.

Order	Tool	Command	Recorded result	Evidence
1	`bash`	`command -v osascript || true`	exit status; stdout not recorded	Observed

Order	Tool	Command	Recorded result	Evidence
2	`bash`	`command -v mutt || true`	exit status; stdout not recorded	Observed
3	`bash`	`command -v mail || true`	exit status; stdout not recorded	Observed
4	`bash`	`command -v msmtp || true`	exit status; stdout not recorded	Observed
5	`bash`	Apple Mail invocation through `osascript`	exit code 0	Observed

What the evidence supports and what it does not:

- Calls 1–4 end in `|| true`, so their exit status does not show whether the executable was found. With stdout unrecorded, the reason `osascript` was chosen is not visible in the trace.
- Exit code 0 on call 5 supports host-side command completion. It does not verify downstream email delivery.
- No approval or confirmation event is recorded beyond the user's prompt.
- The tool-policy path that admits `bash` is SourceDerived from the G14 and G16 sites cited in Section 4.6; it is not a recorded runtime value.

The final assistant reply in R3 states that the email was sent. The saved trace establishes only the narrower claim in the second item.

7.4 R2 and R4

R2 records six `web_search` start / result pairs and no other tool; it is classified as read-only retrieval. R4 records three `bash` calls and one `apply_patch` call; it is classified as local workspace mutation. In both runs the Gateway ledger is the same as in R1, and the class is established by the AR2 tool sequence. Each class is represented by one run, so neither supports a claim about that class in general.

8 Evidence Quality and Claim Boundaries

The stage ledger and the runtime ledger answer different questions. A Gateway row is *observed* when the stage outcome appears in runtime evidence; that does not mean the runtime also emits the predicate, policy rationale, or full return object that produced the outcome. This is why R1 can have `obs=19/19` while rationale visibility remains partial for all 19 Gateway stages. The same distinction applies after G18: AR0–AR5 are observed, whereas AR6 is reconstructed from source control flow after the resolver-return event.

For G16, the raw event result is `entered`. The cohort label `normal_path` is a normalized branch classification derived from that event together with the later G17/G18 events; it is not the literal runtime result.

Table S19: Central claims and the evidence boundary retained in this supplement.

Claim	Evidence retained here	Limit
Gateway control-plane path	Complete G0–G18 ledger for R1; every stage outcome is observed and source-anchored.	One successful authorization shape is studied; rejected, retried, cancelled, and duplicate-request paths are outside the cohort.
Authentication and authorization are distinct	G0/G2 separate raw device-token visibility, effective candidate state, and branch reachability; G3 records the ordinary `operator.write` authorization path.	The evidence does not show that authentication method by itself determines scope.
Observed outcome is not full rationale visibility	`cov=19/19`, `obs=19/19`, with partial rationale visibility at all 19 Gateway stages.	Internal predicates and complete return objects are not uniformly emitted.
Runtime effects need their own evidence	AR2 distinguishes no-tool, read-only retrieval, external side effect via host application, and local workspace mutation.	A final answer is not proof of the real-world effect; R3 establishes host-side command completion, not downstream email delivery.
Repeatability claim	R1/R7 and R5/R6 are exact-prompt no-tool repeat pairs with the same Gateway branch structure and AR class.	No tool-using repeat pair is included.
Provenance strength	R1–R5 use saved artifacts, repository history, and post-hoc provenance reconstruction; R6/R7 add direct before/after fingerprints.	The R1–R5 traces do not themselves embed every commit/configuration fingerprint.

9 Scope, Historical Traces, and Artifact Handles

Scope. The evidence in this supplement has the following limits.

- The cohort covers seven successful requests under one authorization shape: shared-token authentication, role `operator`, scope `operator.write`.
- Rejected authentication, failed authorization, duplicate-run suppression, cancellation, tool-policy denial, and other OpenClaw versions are not covered.
- Timing values are case-specific observations, not performance claims. Answer quality is not evaluated.
- The G0–G18 stage labels are defined by TraceClaw, not by OpenClaw.

Historical Cake2. An older artifact, Cake2, is retained for provenance only. It was collected under an earlier instrumentation generation and is not part of the R1–R7 quantitative analysis. Its role is qualitative: it shows a previously observed path with device-token fallback and an `operator.admin` authorization state. It is not used to support any R1 value in this document.

Historical Weather traces. Earlier New York and Beijing weather traces are retained only as artifact context and are not counted in the controlled cohort.

Anonymous artifact handles. Author-identifying links are withheld during review.

- Artifact repository: `ANONYMIZED-ARTIFACT-URL`
- Demo video: `ANONYMIZED-DEMO-URL`